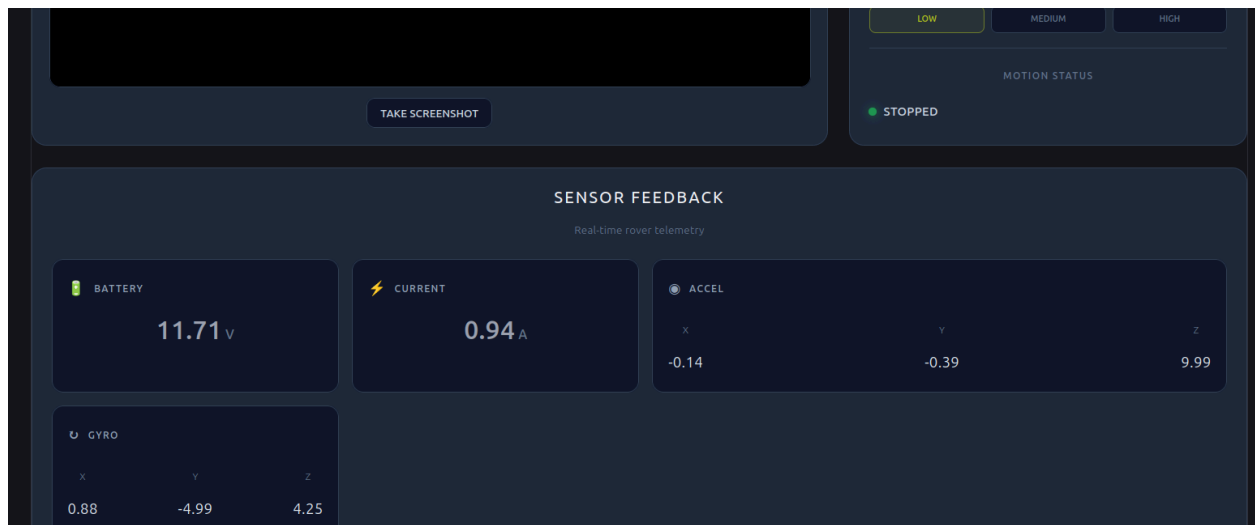
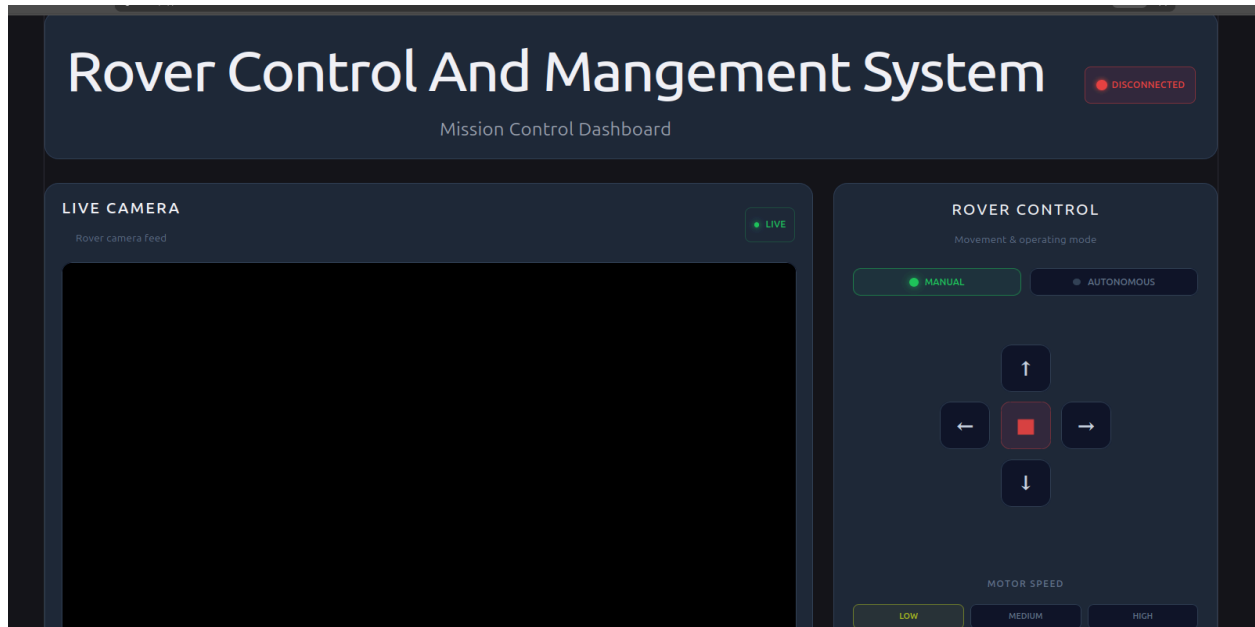


Technical report: React back-end part by Youssef Eslam

GUI:



The GUI was designed as a rover control dashboard where the operator can monitor and control the rover from one interface. The Rover Control panel allows the user to switch between manual and autonomous operation, control the rover's movement, select the motor speed, and monitor the current motion status. The interface also provides a connectivity indicator that changes according to the connection status between the GUI and the rover backend, allowing the operator to immediately recognize communication problems.

The Live Camera section provides a real-time video feed from the rover's webcam. This allows the operator to observe the rover's surroundings while controlling it.

For autonomous operation, an Object Detection sub-window can be provided to display the processed camera feed. The camera frames are processed using the YOLO object-detection model, and detected objects are highlighted on the video. This gives the operator visual feedback about what the autonomous system is detecting while the rover is operating.

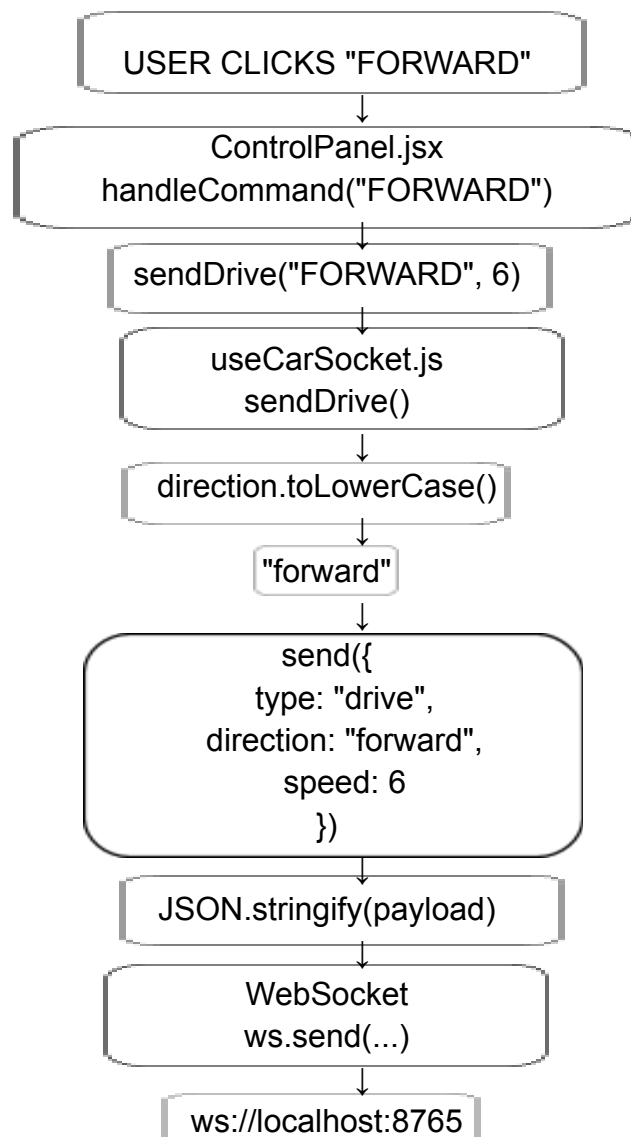
The Sensor Feedback section continuously displays important rover telemetry, including battery voltage, current consumption, accelerometer readings, and gyroscope readings. This allows the operator to monitor the rover's condition in real time. Warning messages are also displayed when unsafe conditions such as low voltage or excessive current are detected.

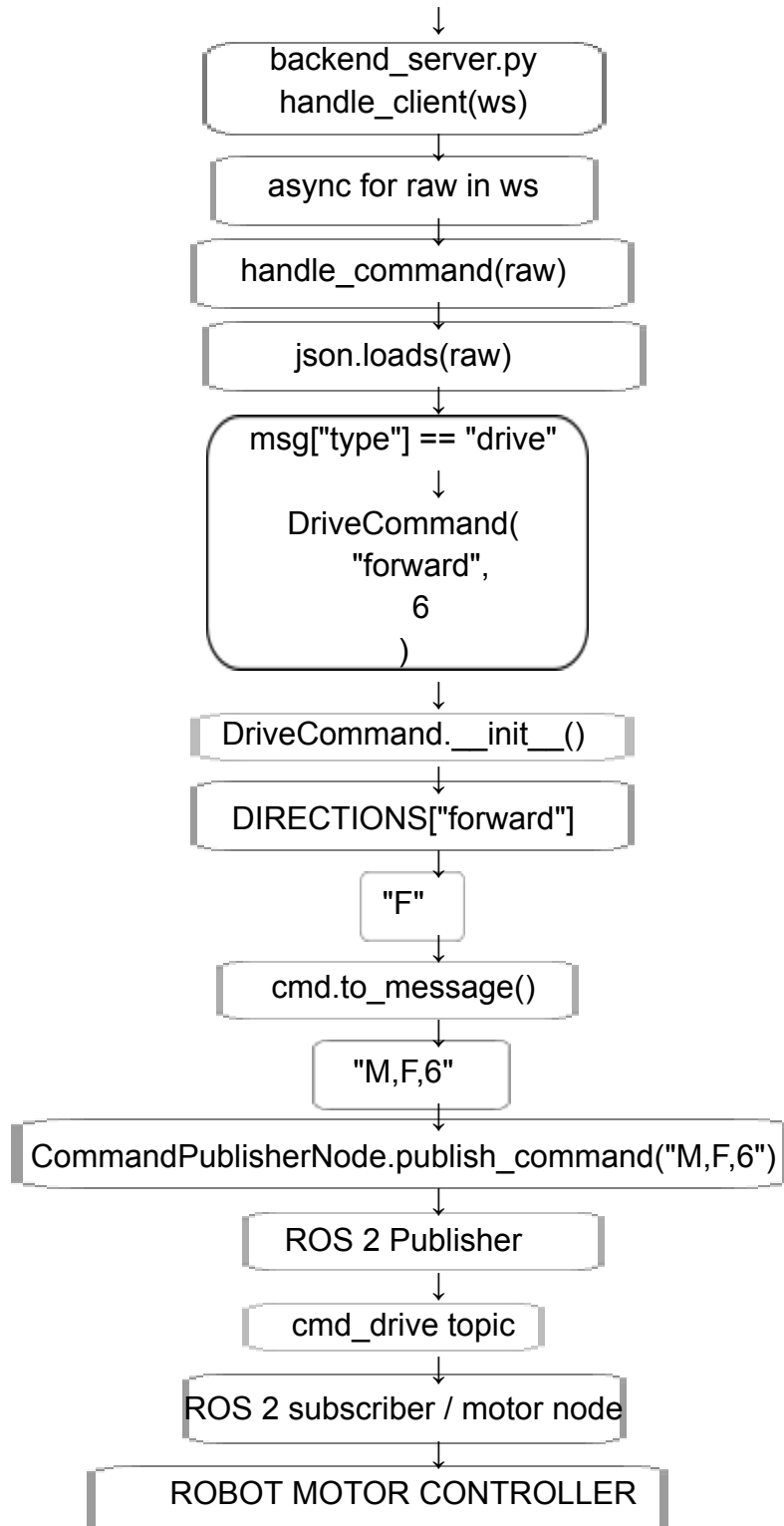
Algorithm and flow:

## 1-motion command

The process starts when the user clicks a movement button, such as forward, and selects a speed such as medium, which has a value of 6. React sends the direction and speed through the WebSocket as a JSON message, for example `{"type":"drive","direction":"forward","speed":6}`. The Python backend receives the message, checks that the direction and speed are valid, and converts the direction into the robot's short command format. Therefore, forward becomes F, producing M,F,6. This command is published as a ROS 2 String on the `cmd_drive` topic. The motor-control node receives it and uses it to move the rover.

flowchart:

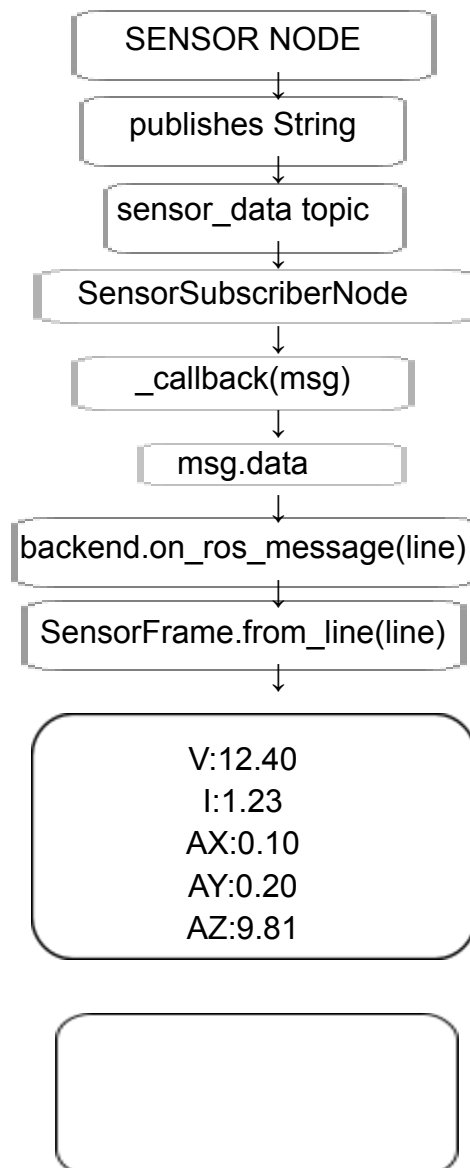




## 2-sensors algorithm :

The sensor node collects voltage, current, accelerometer, and gyroscope data and publishes them as a String on the sensor\_data ROS 2 topic. The Python backend receives the message, separates the values, converts them into numbers, and organizes them into sensor data. It also checks the readings for unsafe conditions, such as voltage below 9.0 V or current above 5.0 A, and generates warnings when necessary. The processed data is converted into JSON, such as `{"type":"telemetry", ...}`, and sent through the WebSocket to React. The React WebSocket layer receives and parses the message, updates the telemetry state, and passes it to `SensorPanel.jsx`, where the sensor values are displayed. This approach allows Python to process and analyze the sensor data before it reaches the GUI.

Flowchart:



GX:0.10

GY:0.20

GZ:0.30



SensorFrame object



analyze\_and\_warn(frame)



check voltage/current



frame.to\_json()



JSON telemetry



broadcast(...)



WebSocket



useCarSocket.js  
ws.onmessage



JSON.parse(event.data)



msg.type === "telemetry"



setTelemetry(...)



telemetry state changes



App.jsx  
telemetry



<SensorPanel telemetry={telemetry} />



SensorPanel.jsx



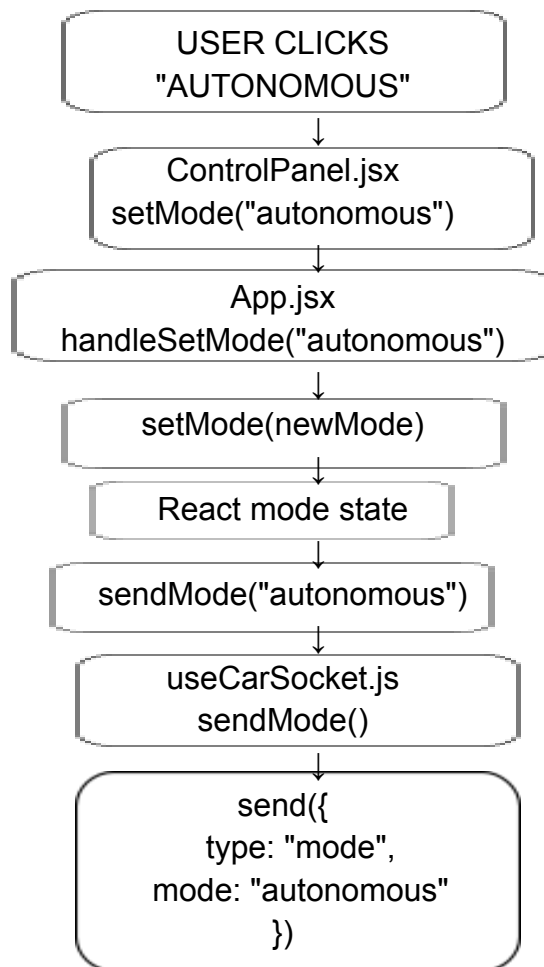
voltage/current/accel/gyro displayed

### 3-mode change

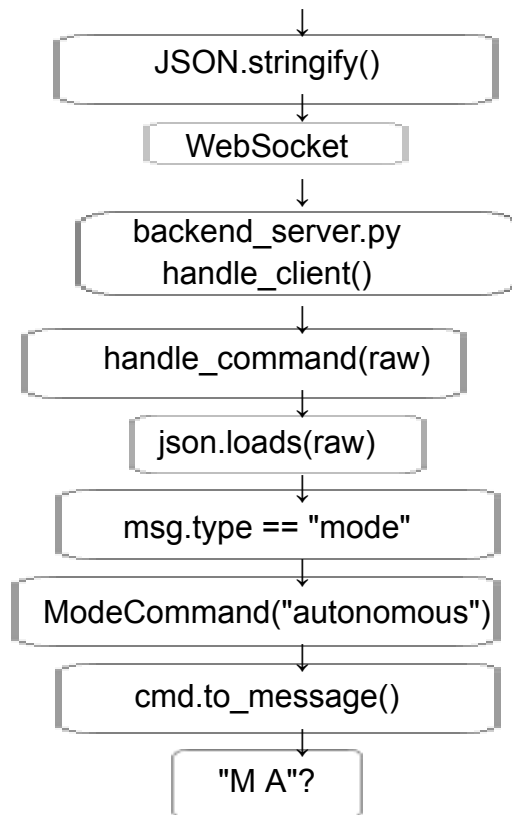
algorithm :

The process starts when the user selects manual or autonomous. React updates its current mode and sends a JSON message through the WebSocket, for example `{"type": "mode", "mode": "autonomous"}`. The Python backend receives the message and converts the selected mode into the robot's command format. autonomous becomes MA, while manual becomes MM. The command changes its operating mode. React also uses the updated mode to control the GUI, such as disabling manual movement buttons during autonomous operation.

Flowchart:





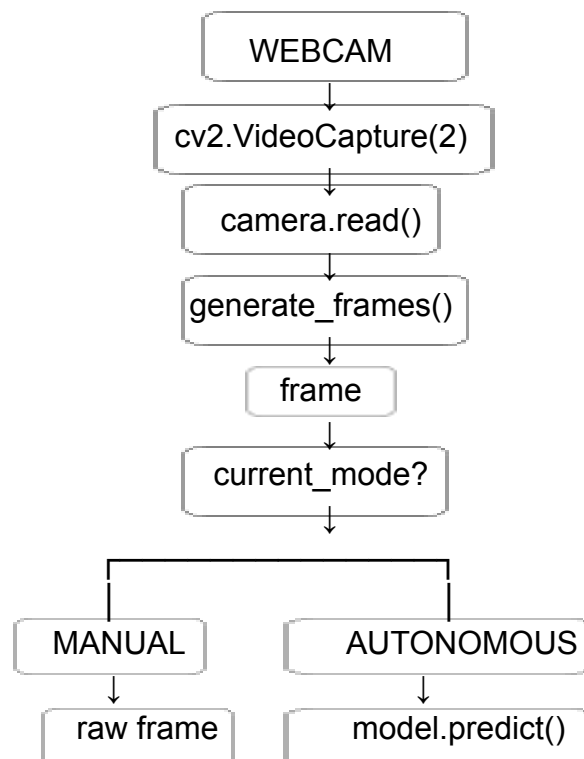


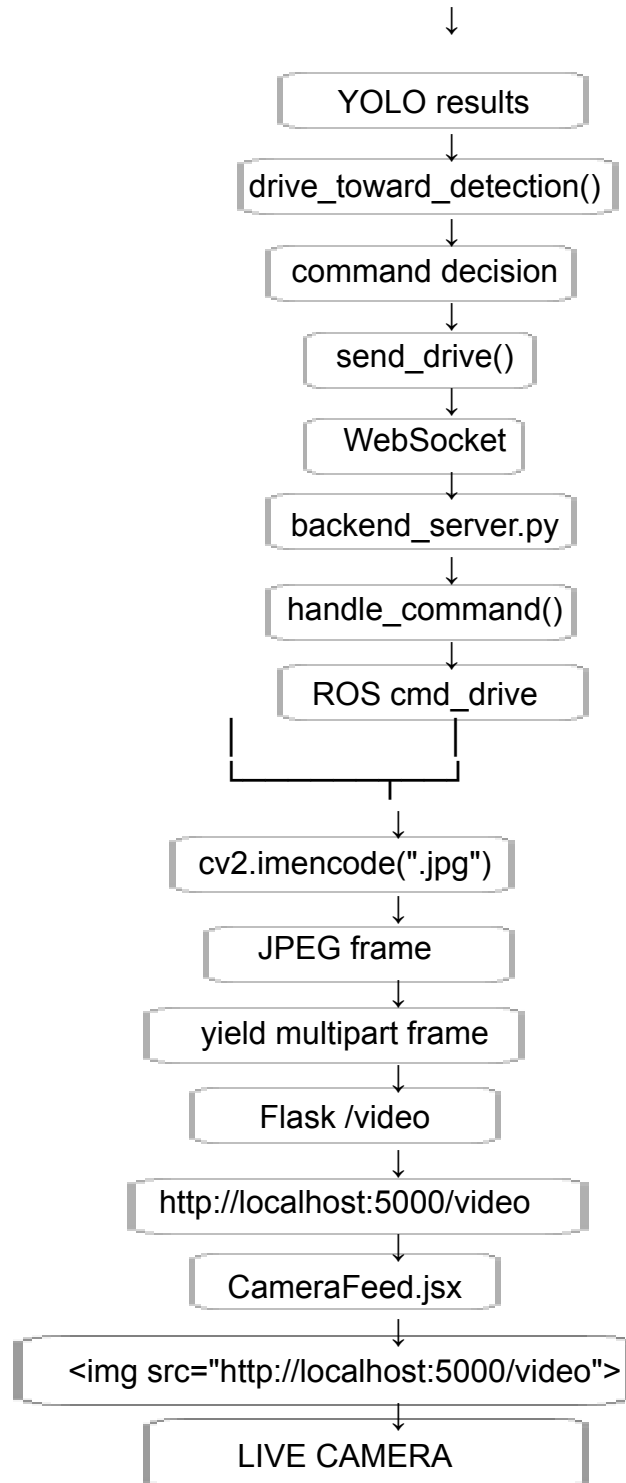
#### 4-camera

##### algorithm :

The camera server continuously captures frames using OpenCV from camera device 2. Each frame is resized to  $640 \times 480$  and the current operating mode is checked. In manual mode, the frame is streamed directly without running YOLO. In autonomous mode, the frame is processed by YOLO to detect the target object. The system checks the object's position and size to decide how the rover should move. If no object is detected or the object is close enough, it sends stop. If the object is centered, it sends forward. If it is to the right, it sends forward\_right, and if it is to the left, it sends forward\_left. These commands are sent through the WebSocket to the backend, converted into commands such as M,F,140, M,FR,140, or M,FL,140, and published to the cmd\_drive ROS 2 topic. At the same time, the processed camera frame is encoded as JPEG and streamed through Flask to CameraFeed.jsx, allowing the GUI to display the live or YOLO-annotated camera feed.

Flowchart:





Note:my documentation also is explaining most of the code the explanation is after #

## Backend\_server:

```
import argparse
#can send values from terminal
import asyncio
#used for asynchronous programing server communicate with multiple clients
import json
#convert between python and json
import logging
#used for messages like print()
import threading
#do multiple tasks
from abc import ABC, abstractmethod
#for oop

import rclpy
from rclpy.node import Node
from rclpy.executors import SingleThreadedExecutor
from std_msgs.msg import String
#ros2 libraries for nodes

import websockets
from websockets.server import WebSocketServerProtocol
#this is for websockets

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s
%(message)s")
log = logging.getLogger("backend")
#control how logging appears

class SensorFrame:
    def __init__(self, voltage, current, accel, gyro):
        self.voltage = voltage
        self.current = current
        self.accel = accel
        self.gyro = gyro
#this class is sensor reading of v , i ,imu
```

```

@classmethod
def from_line(cls, line):
    try:
        parts = dict(p.split(":", 1) for p in line.strip().split(","))
        voltage = float(parts["V"])
        current = float(parts["I"])
        accel = [float(parts["AX"]), float(parts["AY"]),
float(parts["AZ"])]
        gyro = [float(parts["GX"]), float(parts["GY"]),
float(parts["GZ"])]
        return cls(voltage, current, accel, gyro)
#this takes the raw data sensor txt and remove whitespaces and , and
organise them and put them into sensorframe
    except (KeyError, ValueError, IndexError):
        log.warning("Malformed line from ROS topic: %r", line)
        return None

def to_json(self):
    return json.dumps({
        "type": "telemetry",
        "voltage": self.voltage,
        "current": self.current,
        "accel": {"x": self.accel[0], "y": self.accel[1], "z":
self.accel[2]},
        "gyro": {"x": self.gyro[0], "y": self.gyro[1], "z":
self.gyro[2]},
    })
#this convert python txt to json for react for websockets

class MotionCommand(ABC):
    @abstractmethod
    def to_message(self) -> str:
        ...
#every subclass must provide to_message()

```

```

class DriveCommand(MotionCommand):
    DIRECTIONS = {"forward": "F", "backward": "B", "left": "L", "right":
"R", "stop": "S",
                  "forward_left": "FL", "forward_right": "FR",
"backward_left": "BL", "backward_right": "BR"}
#motion letters

    def __init__(self, direction, speed):
        if direction not in self.DIRECTIONS:
            raise ValueError(f"unknown direction: {direction}")
#this is for invalid commands
        if not 0 <= speed <= 255:
            raise ValueError("speed must be 0-255")
#range of speed

        self.direction = direction
        self.speed = speed

    def to_message(self) -> str:
        return f"M,{self.DIRECTIONS[self.direction]},{self.speed}"
#command syntax that get publishes M,direction,speed

class ModeCommand(MotionCommand):
    def __init__(self, mode):
        self.mode = mode

    def to_message(self) -> str:
        return f"M{self.mode[0].upper()}"
#this is to send mode autonomous or manual

class SensorSubscriberNode(Node):
    def __init__(self, on_message, topic: str):
        super().__init__("sensor_subscriber")
        self.on_message = on_message
        self.create_subscription(String, topic, self._callback, 10)
        self.get_logger().info(f"Subscribed to sensor topic: {topic}")

```

```
#this is for creation the node type subscriber that take sensor reading
and send log message of the topic given and call function callback
```

```
def _callback(self, msg: String):
    self.on_message(msg.data)
```

```
#contain sensor reading
```

```
class CommandPublisherNode(Node):
```

```
    def __init__(self, topic: str):
        super().__init__("command_publisher")
        self.publisher_ = self.create_publisher(String, topic, 10)
        self.get_logger().info(f"Publishing commands on topic: {topic}")
```

```
#this creates node for publishing commands to ros node
```

```
    def publish_command(self, text: str):
        msg = String()
        msg.data = text
        self.publisher_.publish(msg)
        self.get_logger().info(f"Published command: {text}")
```

```
#publish data to ros
```

```
class GuiBackendServer:
```

```
    def __init__(self, host="0.0.0.0", ws_port=8765):
```

```
#websockets port 8765
```

```
        self.host = host
        self.ws_port = ws_port
        self.clients = set()
```

```
#store connected clints like camera server
```

```
        self.loop = None
        self.command_node: CommandPublisherNode | None = None
```

```
#this is so backend can publish commands through ros
```

```
    async def register(self, ws):
        self.clients.add(ws)
        log.info("Client connected (%d total)", len(self.clients))
```

```
#register react to clints
```

```
    async def unregister(self, ws):
        self.clients.discard(ws)
```

```

        log.info("Client disconnected (%d total)", len(self.clients))
#remove clints when disconnect

    async def broadcast(self, message: str):
        if not self.clients:
            return
        await asyncio.gather(*(ws.send(message) for ws in self.clients),
return_exceptions=True)
#sends messages for all clients

    def analyze_and_warn(self, frame: SensorFrame):
        if frame.voltage < 9.0:
            return "LOW_VOLTAGE"
        if frame.current > 5.0:
            return "OVERCURRENT"
        return None
#for safety analysis

    def on_ros_message(self, line: str):
        frame = SensorFrame.from_line(line)
#send raw string to sensor frame
        if not frame or self.loop is None:
            return

        asyncio.run_coroutine_threadsafe(self.broadcast(frame.to_json()),
self.loop)
#convert to json and send data to react over websocket
        warning = self.analyze_and_warn(frame)
        if warning:
            asyncio.run_coroutine_threadsafe(
                self.broadcast(json.dumps({"type": "warning", "code":
warning})), self.loop
            )
#send warning to react over websockets to header

    async def handle_client(self, ws):
#recive commands from react
        await self.register(ws)
        try:

```



```

        async for raw in ws:
            await self.handle_command(raw)
    finally:
        await self.unregister(ws)
    async def handle_command(self, raw: str):
        try:
            msg = json.loads(raw)
#change json to python

        except json.JSONDecodeError:
            log.warning("Bad JSON from client: %r", raw)
            return

        if msg.get("type") == "drive":
            cmd = DriveCommand(msg["direction"], msg.get("speed", 3))
#to control motion
        elif msg.get("type") == "mode":
            cmd = ModeCommand(msg["mode"])
#contol mode of car
        else:
            log.warning("Unknown command type: %s", msg.get("type"))
            return

        if self.command_node is not None:
            self.command_node.publish_command(cmd.to_message())

    async def run(self):
#start websocket server
        self.loop = asyncio.get_running_loop()
        async with websockets.serve(self.handle_client, self.host,
self.ws_port):
#Start listening
            log.info("WebSocket server listening on ws://%s:%d", self.host,
self.ws_port)
            await asyncio.Future()
        # run forever

def ros_spin_thread(executor: SingleThreadedExecutor):
    executor.spin()
#runs ros processing

```

```

def main():
    parser = argparse.ArgumentParser()
#create command line config
    parser.add_argument("--ws-port", type=int, default=8765)
    parser.add_argument("--sensor-topic", default="sensor_data")
    parser.add_argument("--cmd-topic", default="cmd_drive")
    args = parser.parse_args()
#the Argument we can change and its default value

    backend = GuiBackendServer(ws_port=args.ws_port)
#create backend

    rclpy.init()
#start ros

    sensor_node = SensorSubscriberNode(backend.on_ros_message,
args.sensor_topic)
    command_node = CommandPublisherNode(args.cmd_topic)
    backend.command_node = command_node
#create sensor and command node
    executor = SingleThreadedExecutor()
    executor.add_node(sensor_node)
    executor.add_node(command_node)
#make executor manege nodes
    ros_thread = threading.Thread(target=ros_spin_thread, args=(executor,),
daemon=True)
#create a thread that its job is to run ros spin
    ros_thread.start()
    try:
        asyncio.run(backend.run())
    except KeyboardInterrupt:
        pass
    finally:
        sensor_node.destroy_node()
        command_node.destroy_node()
        rclpy.shutdown()

```

```
if __name__ == "__main__":  
    main()
```

Camera\_server:

```
import json #this is for converting Python to JSON  
import time  
  
import cv2 #this is for OpenCV that gives Python access to each camera  
frame  
import websocket  
from flask import Flask, Response, request, jsonify  
from flask_cors import CORS  
#for local web server for video for react  
from ultralytics import YOLO  
# this is for detection model  
  
app = Flask(__name__)  
CORS(app)  
#create flask server object and CORS(app) allows for communication through  
diff local host  
  
camera = cv2.VideoCapture(2, cv2.CAP_V4L2)  
#to open camera number 2 with Linux camera  
camera.set(cv2.CAP_PROP_FRAME_WIDTH, 640)  
camera.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)  
camera.set(cv2.CAP_PROP_BUFFERSIZE, 1)  
# It's low resolution of 640x480 and dec camera buffer to reduce latency  
  
if not camera.isOpened():  
    raise RuntimeError("Could not open camera")  
#checks if camera didn't work  
model = YOLO("best.pt")  
#loads the model i trained  
  
FRAME_W, FRAME_H = 640, 480 #dimentions variable  
BACKEND_WS_URL = "ws://localhost:8765" #backend_server local host  
  
current_mode = "manual"  
#variable stores current car mode and starts with manual
```

```

SPEED = 3
DEADZONE = 60      # the object is 60 pixels from the center; it's centered
CLOSE_FILL_RATIO = 0.92  # box fills 92% of the frame it stops
_ws = None # this is for websocket connection to backend_server but didn't
connect yet

def get_ws():
    #this gets the WebSocket connection
    global _ws
#will change this variable global not only inside locally
    if _ws is None:
#if there is no connection
        try:
            _ws = websocket.create_connection(BACKEND_WS_URL, timeout=2)
            _ws.settimeout(0) # connect to back end server
        except Exception as exc:
            print(f"[autonomous] couldn't reach backend: {exc}")
            _ws = None
    return _ws
#if connection fail print it failed

def send_drive(direction, speed=SPEED):
    #the drive command sent to the backend server used by the car
    autonomously

    global _ws
    ws = get_ws()
    if ws is None:
        return

    #get websocket and connect to it
    try:
        while True:
            ws.recv()
    except Exception:
        pass

# The backend always sends data like warnings to web sockets, but here we
don't want it

```

```

    try:
        ws.send(json.dumps({"type": "drive", "direction": direction,
"speed": speed}))
#convert python to json string to be sent through websockets
    except Exception as exc:
        print(f"[autonomous] send failed, reconnecting: {exc}")
#prints error message if fail
        try:
            ws.close()
        except Exception:
            pass
        _ws = None
#and close the websocket

def drive_toward_detection(results):
    boxes = results[0].boxes
#detect box for first image as we give it only a frame at a time
    if boxes is None or len(boxes) == 0:
        send_drive("stop")
        return
#if nothing is detected stop

    best_i = boxes.conf.argmax().item()
#chooses highest confident box
    x1, y1, x2, y2 = boxes.xyxy[best_i].tolist()
#get four point of box
    cx, box_w, box_h = (x1 + x2) / 2, x2 - x1, y2 - y1
#calc center and width and height
    if max(box_w / FRAME_W, box_h / FRAME_H) >= CLOSE_FILL_RATIO:
        send_drive("stop")
        return
#stop when it fill 0.92 of screen

    offset = cx - FRAME_W / 2 #how far center object from camera
    if abs(offset) <= DEADZONE: # if it in center move forward
        send_drive("forward")
    elif offset > 0: #positive move right
        send_drive("forward_right")
    Else: #negative left
        send_drive("forward_left")

```

```

@app.route("/mode", methods=["POST"])
#this is to change car mode
def set_mode():
    global current_mode
#change a variable outside function
    data = request.get_json(silent=True) or {}
#it take the jason datafrom react
    mode = data.get("mode")

    if mode not in ("manual", "autonomous"):
        return jsonify({"error": "mode must be 'manual' or 'autonomous'"}),
400
#if there is an error send this line

    current_mode = mode
    if mode == "manual":
        send_drive("stop")
#when we switch back to manual stop car

    print(f"Mode switched to: {current_mode}")
    return jsonify({"mode": current_mode})
#flask send json mode back to react

def generate_frames():
#generate camera frame to stream

    while True:
        success, frame = camera.read()
        if not success:
            print("Frame read failed")
            break
#if failed send that it failed
        frame = cv2.resize(frame, (FRAME_W, FRAME_H))
#resize frame to 640x480
        if current_mode == "autonomous":
#if we are in autonomous mode use trained model

```

```

        results = model.predict(frame, imgsz=640, conf=0.2,
verbose=False)
#its processed frame from yolo and itslow confidence no problem it moves
to the most conf
        drive_toward_detection(results)
#moves car as we saw up
        frame = results[0].plot()
#create box around object
        success, buffer = cv2.imencode(".jpg", frame,
[cv2.IMWRITE_JPEG_QUALITY, 60])
Change frames to jpg to be faster transmission

        if not success:
            continue
#if frame doesnt work forget it and move to next frame
        yield (
            b"--frame\r\n"
            b"Content-Type: image/jpeg\r\n\r\n"
            + buffer.tobytes()
            + b"\r\n"
        )
        time.sleep(1 / 30)
#at 30 frame it sends video as jpeg frames

@app.route("/video")
def video():
    return Response(generate_frames(), mimetype="multipart/x-mixed-replace;
boundary=frame")
#send this stream

@app.route("/")
def index():
    return "Camera server is running!"

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, threaded=True)

```

usecardocket:

note:Its a reusable piece of code and websocket code its a react hook i can put inside app.jsx

```
import { useEffect, useRef, useState, useCallback } from "react";
#usestate for changes in data and GUI updates and re-rendering
#useeffect for side effects of something component does that isnt what
should appear on screen
#usecallback save function reference
#useref changes things without causing re-rendering

const WS_URL = "ws://localhost:8765";
const WARNING_DISPLAY_MS = 5000;

export function useCarSocket(url = WS_URL) {
  const [connected, setConnected] = useState(false);
#control header
  const [telemetry, setTelemetry] = useState({
    voltage: null,
    current: null,
    accel: { x: 0, y: 0, z: 0 },
    gyro: { x: 0, y: 0, z: 0 },
  });
#stores latest sensor info
  const [warning, setWarning] = useState(null);
#warnings of volt and current

  const wsRef = useRef(null);
  const reconnectTimer = useRef(null);
  const warningTimer = useRef(null);

  const connect = useCallback(() => {
    const ws = new WebSocket(url);
    wsRef.current = ws;
#this is for browser websocket and stores connection

    ws.onopen = () => setConnected(true);
#when websockets connects header say connected
```



```

ws.onclose = () => {
  setConnected(false);
  reconnectTimer.current = setTimeout(connect, 2000);
};
#auto reconnect when connection is lost
ws.onerror = () => ws.close();
#when error happens close and rerun onclose

ws.onmessage = (event) => {
  let msg;
  try {
    msg = JSON.parse(event.data);
  } catch {
    return;
  }
#this handels msg comes fromb ackend

  if (msg.type === "telemetry") {
    setTelemetry({
      voltage: msg.voltage,
      current: msg.current,
      accel: msg.accel,
      gyro: msg.gyro,
    });
#if message is sensor values store them so the sensorpanel can take this
values
  } else if (msg.type === "warning") {
    setWarning(msg.code);
#for header warning
    clearTimeout(warningTimer.current);
    warningTimer.current = setTimeout(() => setWarning(null),
WARNING_DISPLAY_MS);
#show warning for five sec as especfied up and new warning get shown on
top
  }
};
}, [url]);

```

```

useEffect(() => {
  connect();
  return () => {
    clearTimeout(reconnectTimer.current);
    clearTimeout(warningTimer.current);
    wsRef.current?.close();
  };
}, [connect]);
#when component start connect to websocket to backend and after return us
cleanup

const send = useCallback((payload) => {
  if (wsRef.current?.readyState === WebSocket.OPEN) {
    wsRef.current.send(JSON.stringify(payload));
  }
}, []);
#sthis is to send messages to backend through websocket

const sendDrive = useCallback(
  (direction, speed = 3) => send({ type: "drive", direction:
direction.toLowerCase(), speed }),
  [send]
);
const sendMode = useCallback((mode) => send({ type: "mode", mode }),
[send]);
#function so can be used by component for direction and mode

return { connected, telemetry, warning, sendDrive, sendMode };
}

```